

Upskill_Resume

Complete Project Handoff / Context Summary

Purpose: Preserve the current project state, decisions, constraints, and next UI priorities so development can continue in a new ChatGPT conversation without repeating context.

1. PROJECT OVERVIEW

Project name: Upskill_Resume

Purpose: Full-stack AI-powered resume and recruitment platform for students, freshers, entry-level candidates, and job seekers.

Main workflow: User → Upload/Create Resume → AI Resume Analysis → ATS Score + Skills + Skill Gaps + Suggestions → Job Matching → Interview Preparation → Learning Resources.

Target users: College students, freshers, entry-level candidates, job seekers, and placement/interview candidates.

Technology stack:

- Frontend: React + Vite + JavaScript/JSX + CSS
- Backend: Java 21 + Spring Boot 4.1.1 + Spring Web + Spring Data JPA + Spring Security + JWT + Maven
- Database: PostgreSQL 18.6
- AI: Google Gemini API + Google GenAI Java SDK 1.16.0
- Resume processing: Apache Tika 3.2.2
- Tools: VS Code, Antigravity IDE/Agent, Postman, Git/GitHub
- Python is not required for the application.

2. CURRENT UI DEVELOPMENT STATUS

Overall UI completion estimate: approximately **10%** of the intended final UI. This is an approximate estimate, not a measured percentage.

Implemented / existing screens

- **Landing/Home:** Upskill_Resume branding, top navbar, hero, AI-Powered Career Platform badge, heading, description, two primary cards (Create Resume and ATS Check), plus feature/workflow/footer sections.
- **Login:** email/password form, login API integration, JWT storage, authentication-aware ATS redirect.
- **Register:** React registration page connected to authentication backend; further polish may remain.
- **Dashboard:** exists and follows the landing-page concept; navbar, logout, hero, Create Resume and ATS Check. Still mostly a placeholder.
- **ATS / Resume Upload:** most developed UI: drag/drop, browse, PDF/DOCX validation, 5 MB limit, selected-file state, analyze/loading states, ATS score circle, summary, skills, missing skills, suggestions, expandable “See Exactly What To Write”, Analyze Another Resume.
- **Job Matching:** JobMatching.jsx exists; complete integration is not finished.
- **Interview Preparation:** InterviewPreparation.jsx exists; complete integration is not finished.
- **Learning Resources:** LearningResources.jsx exists; backend resource endpoints have been tested and the frontend path was corrected to /api/resources/skill/....
- **Resume Analysis:** ResumeAnalysis.jsx exists; final navigation/integration is incomplete.

Currently working / partially implemented

- Authentication flow is implemented; login returns a plain-text JWT and frontend uses response.text().
- Landing ATS action handles logged-in vs logged-out users and redirects after login.

- ATS upload and analysis UI is connected to the backend flow.
- Learning Resources backend endpoint works based on prior Postman testing.
- Feature pages exist but are not fully integrated/polished.
- ATS detailed suggestion area still has some UI placeholder text; exact generated wording is not fully represented by the backend.

Still missing / incomplete

- Final application-wide navigation wiring.
- Fully polished Job Matching UI and API integration.
- Fully polished Interview Preparation UI and API integration.
- Real Dashboard data and dashboard API integration.
- Final Learning Resources polish.
- About Us and Contact UI.
- Rich structured ATS suggestions with title, problem, what-to-write, and example.
- Complete responsive/mobile polish.
- Consistent loading, empty, validation, error, and unauthorized states.
- Final visual consistency pass and end-to-end UI testing.

3. CURRENT PROJECT STRUCTURE

Root:

```
Upskill_Resume/
  ■■■ backend/
  ■■■ frontend/
  ■■■ .git/
```

Frontend:

```
frontend/
  ■■■ src/
  ■ ■■■ main.jsx
  ■ ■■■ App.jsx
  ■ ■■■ App.css
  ■ ■■■ index.css
  ■ ■■■ api.js
  ■ ■■■ pages/
  ■ ■■■ Login.jsx
  ■ ■■■ Register.jsx
  ■ ■■■ Dashboard.jsx
  ■ ■■■ ResumeUpload.jsx
  ■ ■■■ ResumeAnalysis.jsx
  ■ ■■■ JobMatching.jsx
  ■ ■■■ InterviewPreparation.jsx
  ■ ■■■ LearningResources.jsx
  ■■■ package.json
```

Frontend framework: React + Vite.

Navigation: App.jsx currently uses React state/localStorage page switching rather than a confirmed dedicated routing library. Important states include home, login, register, dashboard, and ats.

Styling: App.css and index.css. No confirmed external UI framework/library is currently used.

API helper: api.js provides apiFetch.

Backend: Spring Boot REST API with controllers, services, entities, repositories, DTOs, and security/configuration.

Database: PostgreSQL database named upskill_resume; JPA/Hibernate; ddl-auto=update.

AI: GeminiService exposes analyzeResume, matchResumeWithJob, and generateInterviewQuestions.

Implemented backend endpoints

- POST /api/users/register

- POST /api/users/login
- POST /api/resumes/upload
- GET /api/resumes/my
- POST /api/resumes/analyze/{resumeId}
- GET /api/resumes/analysis/{resumeId}
- GET /api/resources
- GET /api/resources/skill/{skill}

Planned / partially implemented APIs

- POST /api/job-matching/{resumeId}
- POST /api/interview/{resumeId}
- GET /api/dashboard

4. DESIGN DIRECTION

Visual style: Modern, professional AI career/recruitment platform; substantially more polished and feature-rich than a basic college CRUD project.

Reference: 1millionresume.com was discussed as inspiration, but Upskill_Resume should be more feature-rich.

Homepage hierarchy: strong hero → two prominent actions: Create Resume and ATS Check → supporting feature/workflow/content sections.

Colors/theme: exact finalized palette was not formally specified. Preserve the existing CSS rather than inventing a new palette without instruction.

Typography: exact finalized font family was not formally specified. Preserve existing hierarchy and spacing.

Layout: wide responsive landing layout, top navigation, hero, card-based areas, dedicated feature pages, responsive CSS.

Design patterns: navbar, hero badge, feature cards, action buttons, upload/drop zone, score visualization, analysis cards, skill tags, missing-skill tags, expandable suggestions, loading spinner.

ATS score: CSS circular/conic-gradient visualization.

Responsive: ATS responsive CSS has started; final responsive polish remains.

5. DECISIONS ALREADY MADE

- Keep project name Upskill_Resume.
- College project; relatively low expected load.
- React + Spring Boot + PostgreSQL + REST + JWT + hosted Gemini.
- No Ollama/local LLM and no model training.
- Apache Tika for PDF/DOCX extraction.
- JWT authentication and BCrypt password hashing.
- Protected endpoints require authentication; user-specific resources require ownership checks.
- Homepage has only two large primary actions: Create Resume and ATS Check.
- Navbar: Home, Resume Analysis, Job Matching, Interview Preparation, Learning Resources, About Us, Contact.
- No separate Dashboard navbar item; Home/Dashboard share the same concept.
- Do not visibly expose Resume IDs to normal users.
- ATS accepts PDF/DOCX with a 5 MB maximum.
- ATS shows score, summary, skills, missing skills, and practical suggestions.
- Detailed suggestion direction is "See Exactly What To Write".

- Never commit API keys, passwords, or JWT secrets.
- Develop incrementally; do not replace the whole application for individual features.
- User prefers one concrete step/prompt at a time, concise instructions, and testing after meaningful milestones rather than every tiny change.

6. NEXT UI WORK

- 1. Finish global navbar/navigation.
- 2. Finish Job Matching UI + integration.
- 3. Finish Interview Preparation UI + integration.
- 4. Upgrade ATS structured suggestions and “See Exactly What To Write”.
- 5. Build real Dashboard UI using dashboard API.
- 6. Polish Learning Resources.
- 7. Build About Us and Contact.
- 8. Complete responsive/mobile behavior.
- 9. Add consistent loading/empty/error/unauthorized states.
- 10. Final visual polish and end-to-end browser testing.

7. ANTIGRAVITY INSTRUCTIONS

Antigravity should inspect the actual repository before editing and implement one feature at a time.

Preserve

- Existing architecture/package structure.
- Authentication/JWT implementation.
- Existing API helper and auth flow.
- Landing-page and ATS visual language.
- Existing CSS and responsive work.
- Working endpoints and resume upload flow.
- Project name and existing page components.

Do not change unnecessarily

- Do not rewrite the entire frontend/backend.
- Do not replace working APIs with unrelated architectures.
- Do not redesign the visual system from scratch.
- Do not change security unless required.
- Do not introduce unnecessary libraries.
- Do not expose secrets.
- Do not display internal Resume IDs.
- Do not create duplicate pages.
- Do not modify unrelated files.

Coding conventions

- Use existing React functional components/hooks.
- Use `apiFetch` for frontend API calls.
- Keep backend layered as controller/service/repository/entity/DTO.

- Use Authentication to identify logged-in users.
- Enforce ownership on user-specific resources.
- Keep errors user-facing and avoid leaking stack traces/secrets.
- Compile/test relevant work after implementation.
- Report exactly which files changed.
- Avoid broad refactors unless explicitly requested.

8. IMPORTANT CONTEXT

Environment

- Java 21
- Node 24.12.0
- npm 11.6.2
- Git 2.52.0
- PostgreSQL 18.6
- Spring Boot 4.1.1
- Frontend: `http://localhost:5173`
- Backend: `http://localhost:8080`

Local paths

```
Backend:
C:\Users\nikhi\OneDrive\Desktop\JavaRoadmap\Projects\Upskill_Resume\backend

Frontend:
C:\Users\nikhi\OneDrive\Desktop\JavaRoadmap\Projects\Upskill_Resume\frontend
```

Security/auth

- SecurityConfig uses stateless sessions, disabled CSRF, centralized CORS, and JWT filter.
- Public endpoints are registration/login; other endpoints require authentication.
- JwtService signs tokens with the configured secret and uses email as subject.
- JwtAuthenticationFilter reads Authorization: Bearer and sets authenticated email.
- ResumeService includes `getResumeByIdAndUser(...)` for ownership checking.

Gemini quota context

- The existing Gemini project previously returned 429 quota-exceeded errors for gemini-3.6-flash.
- AI Studio showed approximately RPM 4/5, TPM 192.68K/250K, RPD 13/20.
- A new project/key was investigated, but its rate-limit screen showed only AntigraVity and no Gemini text-generation model. Do not substitute it into GeminiService unless Gemini API access is confirmed.
- Do not repeatedly call Gemini during development; preserve quota for meaningful tests.

Git/versioning

- GitHub repository: `Nikhil-tech-eng/Upskill_Resume`; repository is public.
- Secrets were previously exposed in Git history; DB password was changed and history rewritten.
- A Version 1 milestone/upload was intentionally preserved.
- A later checkpoint was committed as "Version 2 - Authentication and AI Features" with requested timestamp 26 Aug 2026, 7:30 PM IST.

AI behavior

- `analyzeResume` requests JSON with `atsScore`, `skills`, `missingSkills`, `suggestions`, `summary`.

- matchResumeWithJob requests matchScore, matchedSkills, missingSkills, recommendations.
- generateInterviewQuestions requests 10 numbered questions.
- Previously observed generation times were about 33 seconds for resume analysis and 53 seconds for interview generation; this was accepted for the college project and can be optimized later.

Uncertainty

The exact current filesystem may differ from the last code excerpts. AntigraVity should inspect the actual repository before edits, especially App.jsx, App.css, page implementations, and the backend tree.

READY-TO-PASTE CONTEXT FOR NEW CHATGPT

I am continuing my college project in this chat. Project name: Upskill_Resume. Do not ask me to repeat the existing project context. Upskill_Resume is a full-stack AI-powered resume/recruitment platform for students, freshers, entry-level candidates and job seekers. Main features: registration/login, JWT auth, resume upload (PDF/DOCX, max 5 MB), Apache Tika extraction, AI ATS analysis, skills, skill gaps, suggestions, "See Exactly What To Write", job-description matching, interview preparation, learning resources, resume history and dashboard. Future resume builder functionality is planned. Stack: React + Vite + JS/JSX + CSS; Java 21 + Spring Boot 4.1.1 + Spring Web + JPA + Spring Security + JWT + Maven; PostgreSQL 18.6; Gemini API via Google GenAI Java SDK 1.16.0; Apache Tika 3.2.2. Tools: VS Code, Antigravity, Postman, Git/GitHub. No Ollama/local LLM and no model training. Paths: Backend: C:\Users\nikhil\OneDrive\Desktop\JavaRoadmap\Projects\Upskill_Resume\backend Frontend: C:\Users\nikhil\OneDrive\Desktop\JavaRoadmap\Projects\Upskill_Resume\frontend Frontend: http://localhost:5173 Backend: http://localhost:8080 GitHub: Nikhil-tech-eng/Upskill_Resume UI is approximately 10% complete. Existing screens/components: Landing/Home, Login, Register, Dashboard placeholder, ResumeUpload/ATS, ResumeAnalysis, JobMatching, InterviewPreparation, LearningResources. Landing has Upskill_Resume branding, navbar, hero, AI-Powered Career Platform badge, "Build Your Career With AI", and only two primary cards: Create Resume and ATS Check. ATS handles auth redirect. ATS UI has drag/drop, PDF/DOCX, 5 MB limit, file state, loading, ATS score circle, summary, skills, missing skills, suggestions, expandable "See Exactly What To Write", Analyze Another Resume. Job Matching, Interview, Learning Resources and Resume Analysis exist but are not fully integrated/polished. Dashboard is mostly placeholder. About Us and Contact are unfinished. Final responsive polish and global UI states are missing. Navbar: Home, Resume Analysis, Job Matching, Interview Preparation, Learning Resources, About Us, Contact. Logged out: Login/Register. Logged in: Logout. No separate Dashboard nav item. Frontend files: src/main.jsx, App.jsx, App.css, index.css, api.js, pages/Login.jsx, Register.jsx, Dashboard.jsx, ResumeUpload.jsx, ResumeAnalysis.jsx, JobMatching.jsx, InterviewPreparation.jsx, LearningResources.jsx. App.jsx currently uses React state/localStorage page switching rather than a confirmed routing library. Important states include home/login/register/dashboard/ats. api.js provides apiFetch. No confirmed external UI framework is used. Backend currently has auth endpoints POST /api/users/register and POST /api/users/login; resume endpoints POST /api/resumes/upload, GET /api/resumes/my, POST /api/resumes/analyze/{resumeld}, GET /api/resumes/analysis/{resumeld}; learning endpoints GET /api/resources and GET /api/resources/skill/{skill}. Partially planned/implemented APIs: POST /api/job-matching/{resumeld}, POST /api/interview/{resumeld}, GET /api/dashboard. Security: JWT + BCrypt. Protected endpoints require auth. User-specific resources must enforce ownership. CORS allows http://localhost:5173. Secrets stay in local .env and never go to GitHub. Design: modern professional AI career/recruitment platform. 1millionresume.com was discussed as inspiration, but Upskill_Resume should be more feature-rich. Preserve existing CSS/design language: wide landing layout, hero, cards, navbar, upload/drop zone, score visualization, analysis cards, skill tags, missing-skill tags, expandable suggestions, responsive CSS. Exact final colors/font were not formally fixed; preserve current styles rather than inventing a new design system. Decisions: keep Upskill_Resume name; React + Spring Boot + PostgreSQL + JWT + hosted Gemini; no Ollama/training; PDF/DOCX max 5 MB; no visible Resume IDs; homepage only two primary actions Create Resume and ATS Check; navbar as above; no Dashboard nav item; detailed ATS suggestions should eventually contain problem, exact wording to write, and example; preserve existing architecture/auth/API/CSS/pages; no unnecessary libraries/duplicate pages; never expose secrets. Next UI priority: 1. Global navbar/navigation. 2. Job Matching UI + integration. 3. Interview Preparation UI + integration. 4. Structured ATS suggestions and "See Exactly What To Write". 5. Real Dashboard UI/API. 6. Learning Resources polish. 7. About Us and Contact. 8. Responsive/mobile. 9. Loading/empty/error/unauthorized states. 10. Final visual polish and browser testing. Antigravity: inspect actual repository first. Preserve architecture, auth/JWT, apiFetch, existing landing/ATS design, CSS, working endpoints and page components. Do not rewrite the application, redesign from scratch, introduce unnecessary libraries, duplicate pages, expose secrets, or modify unrelated files. Use React functional components/hooks, apiFetch, layered backend architecture, Authentication for identity, and ownership checks. Implement one feature at a time and report changed files. Gemini context: previous Gemini 3.6 Flash calls hit 429 limits. AI Studio showed about RPM 4/5, TPM 192.68K/250K, RPD 13/20. A new project/key showed only Antigravity and no Gemini model, so do not use it for the Java Gemini API unless Gemini access is confirmed. Avoid repeated AI calls during development. GitHub is public. Secrets were previously exposed in history; DB password was changed and history rewritten. Version 1 milestone was preserved. Version 2 was committed as "Version 2 - Authentication and AI Features" with requested timestamp 26 Aug 2026, 7:30 PM IST. Important: the exact current filesystem may differ from old excerpts. Inspect the actual repository before making changes. The user prefers concise, step-by-step instructions, one concrete task/prompt at a time, with testing after meaningful milestones rather than every tiny change.